

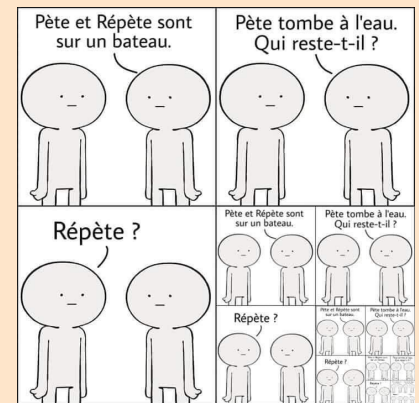


1. Généralités

■ DÉFINITION

La **récursivité** est une démarche qui fait référence à l'objet même de la démarche à un moment du processus. En d'autres termes, c'est une démarche dont la description mène à la répétition d'une même règle.

Un objet **récuratif** est un objet qui, dans sa définition, se sert de lui-même.



■ DÉFINITION

Une fonction sera dite récursive lorsqu'elle s'appelle elle-même.

L'utilité d'une telle fonction sera souvent de se ramener à un problème similaire mais de taille plus petite :

- la fonction va s'appeler elle-même avec un paramètre « plus petit » ;
- cet appel va en générer un autre, puis un autre, etc... ;
- la taille du problème va ainsi diminuer.

Un **risque est que ces appels successifs soient sans fin**. Il est donc nécessaire d'identifier un problème élémentaire que l'on sait résoudre et indiquer que ce problème élémentaire est la **condition d'arrêt** des appels récursifs.

Exercice 1

Essayez de prévoir le résultat de l'exécution du programme ci-dessus. Vérifiez votre hypothèse en exécutant le programme :

```
def fonct(n):  
    if n>0:  
        fonct(n-1)  
    print(n)
```

```
fonct(3)
```

Exercice 2

Quel peut être le rôle de la fonction ci-dessous ?

```
def mystere(L, Acc = []):  
    if L == []:  
        return Acc  
    else:  
        elt = L.pop(0)  
        if elt not in Acc:  
            Acc.append(elt)  
        return mystere(L, Acc)
```

```
print(mystere([3, 5, 3, 4, 1, 2, 1, 3, 9]))
```

■ DÉFINITION : Terminaison d'une fonction récursive

Comment prouver qu'une fonction récursive se termine ?

Cette question se ramène souvent à prouver que les appels successifs ne sont pas sans fin.

Comme en classe de première, il faudra utiliser la notion de **variant de boucle** pour justifier qu'une fonction récursive se termine.

Remarque : En classe de Première et Terminale, on peut par exemple utiliser la définition suivante : On appelle variant d'une boucle toute quantité v telle que :

- $v \in \mathbb{N}$
- v décroît strictement à chaque passage dans la boucle.

2. Ordre des appels

Remarque : Il est important de comprendre que chaque appel récursif met « en pause » l'exécution en cours, en attente d'obtenir le résultat qui est déterminé par l'appel suivant. Concrètement :

- Les appels sont tour à tour mis « en pause » jusqu'au dernier appel qui fournit un résultat. On appelle cela le **dépliage** (ou la **descente**).
- Ce résultat est ensuite transmis à l'appel précédent qui l'utilise pour calculer son propre résultat et le transmettre à l'appel précédent, et ainsi de suite jusqu'au premier appel qui peut alors calculer le résultat final. On appelle cela l'**évaluation** (ou la **remontée**).

Pour illustrer les exemples de cette partie, nous allons utiliser la fonction `rectangle()` définie comme suit :

```
from turtle import *

def rectangle(n):
    """
    n - entier strictement positif
    """
    hideturtle()
    speed(0)
    color('white', 'blue')
    begin_fill()
    for _ in range(2):
        forward(10)
        left(90)
        forward(n)
        left(90)
    end_fill()
    forward(10)
```

La ressource peut être téléchargée [ici](#)

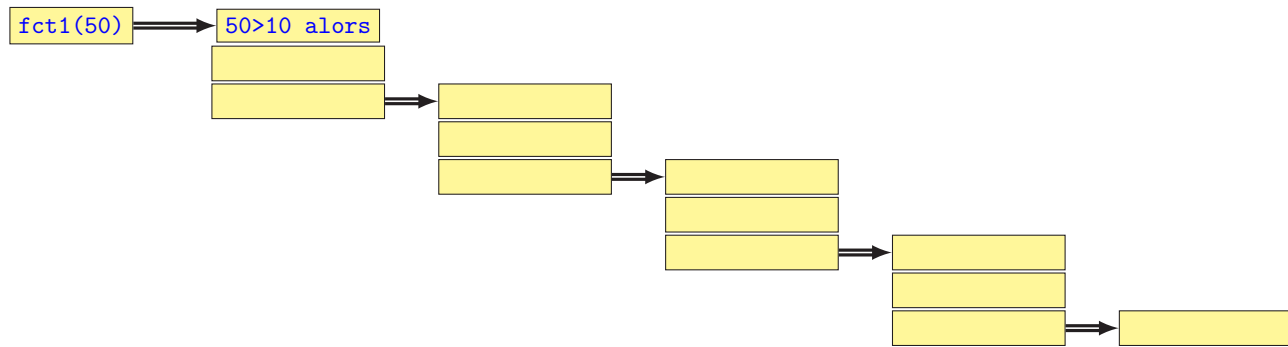
Dans une fonction récursive, l'ordre d'exécution des instructions dépend des appels récursifs à la fonction. En d'autres termes, certaines instructions peuvent être exécutées immédiatement tandis que d'autres sont « mises en attente » dans une pile d'appels.

Exercice 3

On considère la fonction récursive suivante :

```
1 def fct1(n):
2     if n > 10:
3         rectangle(n)
```

On effectue l'appel `fct1(50)`. Complétez la pile d'appel puis tracer la figure correspondante :

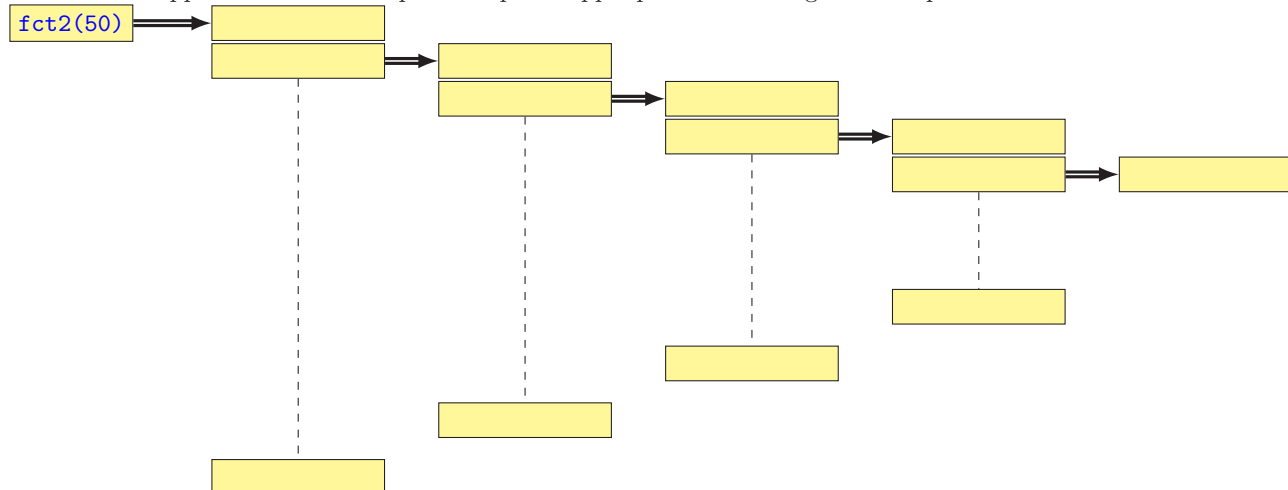


Exercice 4

On échange les instructions des lignes 3 et 4 pour définir la fonction récursive suivante :

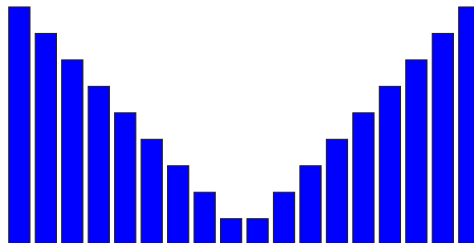
```
1 def fct2(n):
2     if n > 10:
3         fct2(n-10)
4         rectangle(n)
```

On effectue l'appel `fct2(50)`. Complétez la pile d'appel puis tracer la figure correspondante :



Exercice 5

A l'aide de la fonction présentée ci-dessus, définissez une fonction récursive permettant de tracer la figure ci-dessous :



Exercice 6

On considère la fonction récursive suivante :

```
def fct4(n):
    if n > 10:
        fct4(n-10)
```

```
rectangle(n)
fct4(n-10)
```

Réaliser, à la main, le dessin obtenu avec l'appel `fct4(50)`. Sur votre feuille, un carreau représente 10 pixels

3. Complexité

Pour les fonctions récursives envisagées ci-dessous, la complexité en temps est proportionnelle au nombre d'appels récursifs.

Exercice 7

On considère la fonction récursive suivante :

```
def descente(n):
    if n > 0:
        descente(n-1)
```

- 1) Combien d'appels à la fonction `descente()` sont réalisés lorsqu'on exécute l'instruction `descente(5)` ?
- 2) Combien d'appels à la fonction `descente()` sont réalisés lorsqu'on exécute l'instruction `descente(8)` ?
- 3) Donnez, en fonction de `n`, le nombre d'appels à la fonction `descente()` réalisés lorsqu'on exécute l'instruction `descente(n)`.
- 4) En déduire la complexité de cette fonction.

Exercice 8

On considère la fonction récursive suivante :

```
def piano(n):
    if n > 0:
        piano(n-1)
        piano(n-1)
```

- 1) Combien d'appels à la fonction `piano()` sont réalisés lorsqu'on exécute l'instruction `piano(3)` ?
- 2) Combien d'appels à la fonction `piano()` sont réalisés lorsqu'on exécute l'instruction `piano(5)` ?
- 3) Donnez, en fonction de `n`, le nombre d'appels à la fonction `piano()` réalisés lorsqu'on exécute l'instruction `piano(n)`.
- 4) En déduire la complexité de cette fonction.